NEMOX AI RICO-7

MODULE AND DEVELOPER GUIDE | SYNTX FORM

Practical operating and extension manual for the exact preserved unified software package.

COMPANION TO THE COMPLETE TECHNICAL ARCHITECTURE | DATE: 1 NOVEMBER 2026

CONFIDENTIAL | PROPRIETARY | NO USE WITHOUT WRITTEN APPROVAL

This document and the module structure, SYNTX representation, operating and extension guidance it contains are the confidential work of Ottavio Braun, SYNTX SYSTEM. No right of use is granted. Any use, reproduction, disclosure, adaptation, implementation or AI training — in whole or in part — requires the prior, explicit, written approval of Ottavio Braun for a named purpose. Receipt or access creates no licence. Full terms: final page.

NUTZUNG, WEITERGABE ODER UMSETZUNG – GANZ ODER TEILWEISE – NUR NACH AUSDRÜCKLICHER SCHRIFTLICHER FREIGABE DURCH OTTAVIO BRAUN.

STATUS SEMIOTICS ● IMPLEMENTED / VERIFIED ◐ DERIVED ◌ CONCEPTUAL ⊕ EXTENSION ◆ GAP / NOT VERIFIED

01 START AND INSPECT

§1 · LOCAL START · DOCKER ALTERNATIVE

◆ Unpack the preserved package, start both services with one script and open the browser; Docker Compose is the alternative path.

SHELL

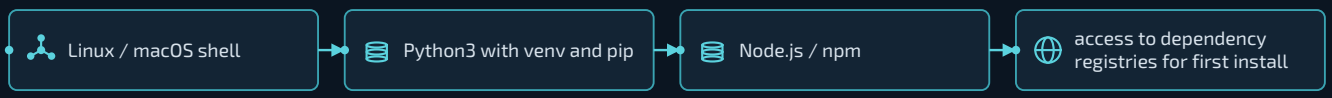
```
› unzip NEMOX_AI_RICO7_COMPLETE_SOFTWARE.zip
› cd nemox-developer-tools
› ./start.sh
# Browser: http://localhost:3000
# Backend health: http://localhost:8000/api/health

# Alternative:
› docker compose up --build
```

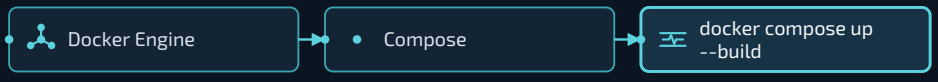
START PATH



PREREQUISITES



DOCKER ALTERNATIVE



◆ PACKAGE BOUNDARY

The ZIP includes source, not preinstalled node_modules or Python virtualenv.

© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

02 MODULE-BY-MODULE INVENTORY

§2 · 29 MODULES · SOURCE ORDER

◆ Every module of the package with its architecture layer and line count. Modules with source-level behaviour carry declared functions, imports, inputs, outputs, writes and extension paths.

Configuration modules (dashed-circle marker below) carry one identical note in the source: Inspect this file for exact configuration and runtime dependencies. No additional runtime behavior is attributed beyond its actual source contents.

README.md	DOCUMENTATION / CONFIG	LINES	47
backend/Dockerfile	RUNTIME	LINES	8
backend/api/__init__.py	API / ORCHESTRATION	LINES	0

backend/api/main.py

API / ORCHESTRATION · LINES 47

DECLARED	Request, Decision, Correction, Confirmation, Subsequent, Answers, invalid, health, intent, decision, feedback, confirmation, next_decision, decision_history, gaps, gap_resolution, graph, reset.
IMPORTS	fastapi, fastapi.middleware.cors, feedback_engine.service, fieldcore.core, gap_detector.service, intent_inspector.service, pydantic, typing.
INPUTS	Pydantic request models.
OUTPUTS	JSON HTTP responses.
EFFECTS	delegates to services; init() at import.
EXTEND	with a new Pydantic model and route and associated API test.

backend/feedback_engine/__init__.py	SOLUTION B	LINES	0

backend/feedback_engine/service.py

SOLUTION B · LINES 49

DECLARED	decide, correct, confirm, retrieve, subsequent, history.
IMPORTS	fieldcore.core.
INPUTS	decision/action/context; feedback correction; confirmation; scope/operation/route.
OUTPUTS	persisted decisions and feedback history.
WRITES	decisions, feedback, confirmed Feedback FieldRecords.
EXTENSION	atomic confirm transaction, explicit FieldRelation edges, validity-aware retrieval.

backend/fieldcore/__init__.py	SHARED FIELD CORE	LINES	0

backend/fieldcore/core.py

SHARED FIELD CORE · LINES 88

DECLARED	now, uid, connection, init, FieldRepository, FieldContextResolver, create, get, transition, relation, graph, reset, resolve.
IMPORTS	contextlib, datetime, pathlib.
INPUTS	field type, scope, JSON payload, source refs, transition target or relation endpoints.
OUTPUTS	JSON-decoded FieldRecord, graph, state changes.
WRITES	fields, relations, transitions.
EXCEPTIONS	ValueError on missing field, scope mismatch, invalid transition or version conflict.
EXTENSION	add schema constraints and a transaction boundary for multi-table operations.

backend/gap_detector/__init__.py	SOLUTION C	LINES	0

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

backend/gap_detector/service.py

SOLUTION C · LINES 24

DECLARED	capability, analyze, evaluate, resolve.
IMPORTS	fieldcore.core.
INPUTS	request text, scope, values, clarification answers.
OUTPUTS	missing/frontier/questions/complete/edges.
WRITES	Request FieldRecord, gaps values_json.
EXTENSION	validate dependency DAG, preserve answer provenance and create explicit gap/context field relations.

backend/intent_inspector/__init__.py · SOLUTION A · LINES 0

backend/intent_inspector/service.py

SOLUTION A · LINES 19

DECLARED	inspect.
IMPORTS	fieldcore.core.
INPUT	request string and scope.
OUTPUT	structured requirements, candidates, unresolved, trace.
WRITES	four FieldRecords, three relations, three transitions.
EXTENSION	replace regex and fixed HISTORY with a source-aware extraction and shipment repository.

backend/requirements.txt · DOCUMENTATION / CONFIG · LINES 5

backend/tests/test_system.py

AUTOMATED TESTS · LINES 36

DECLARED	clean, test_intent_trace, test_feedback_scope_and_confirm, test_gap_resolution, test_state_validation, test_relation_scope.
IMPORTS	api.main, fastapi.testclient, fieldcore.core.
COVERAGE	Five FastAPI TestClient tests use FieldRepository.reset fixture. Test coverage is focused on backend domain paths; frontend and browser paths are not covered.

docker-compose.yml · RUNTIME · LINES 10
frontend/Dockerfile · FRONTEND · LINES 6
frontend/index.html · FRONTEND · LINES 1
frontend/package.json · FRONTEND · LINES 1

frontend/src/components/FieldGraph.tsx

FRONTEND · LINES 8

INPUTS	data.fields and data.relations.
OUTPUT	React Flow nodes and edges.
WRITES	No writes.
EXTEND	with layouts and relation-type legend.

frontend/src/main.tsx

FRONTEND · LINES 13

INPUTS	textarea values, button events, API JSON.
OUTPUTS	three tab views, JSON inspection, FieldGraph.
STATE	local React useState.
EXTENSION	extract typed pages, render dedicated dependency/feedback graphs, add UI tests.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

frontend/src/services/api.ts	FRONTEND	LINES	1

INPUTS	API path and optional JSON body.
OUTPUT	parsed JSON or thrown Error.
PERSISTENCE	No persistence.
EXTEND	with typed responses, timeout and structured errors.

frontend/src/style.css	FRONTEND	LINES	1
frontend/tsconfig.json	FRONTEND	LINES	1
frontend/vite.config.ts	FRONTEND	LINES	2
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/Dockerfile	SOLUTION A	LINES	8
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/Dockerfile	SOLUTION B	LINES	8
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/Dockerfile	SOLUTION C	LINES	8
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/docker-compose.yml	RUNTIME	LINES	10
start.sh	RUNTIME	LINES	17

03 API OPERATIONS WITH EXAMPLES

§3 · 11 OPERATIONS · BACKEND/API/MAIN.PY

◆ Each operation with its implementation path, request and response contract, and the exact curl example from the source where one is given.

GET /api/health

backend/api/main.py → health()

REQUEST	none
RESPONSE	{ok:true}

POST /api/intent

backend/api/main.py → inspect()

REQUEST	{text:str,scope?:str,values?:object}
RESPONSE	request,requirements,candidates,unresolved,field_ids,trace

```
› curl -s -X POST http://localhost:8000/api/intent -H "Content-Type: application/json" -d {"text": "Same shipment as last week, but two days earlier."}
```

POST /api/decisions

backend/api/main.py → decide()

REQUEST	{scope?:str,action:str,context?:object}
RESPONSE	id,scope,action,context

```
› curl -s -X POST http://localhost:8000/api/decisions -H "Content-Type: application/json" -d {"scope": "demo"}
```

POST /api/feedback

backend/api/main.py → correct()

REQUEST	{scope?:str,decision_id:str,correction:object}
RESPONSE	id,decision_id,scope,correction,status

```
› curl -s -X POST http://localhost:8000/api/feedback -H "Content-Type: application/json" -d {"scope": "demo"}
```

POST /api/feedback/{fid}/confirm

backend/api/main.py → confirm()

REQUEST	{scope?:str,accept:bool}
RESPONSE	id,status,field

```
› curl -s -X POST http://localhost:8000/api/feedback/FEEDBACK_ID/confirm -H "Content-Type: application/json" -d {"scope": "demo"}
```

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

POST /api/subsequent — backend/api/main.py → subsequent()

REQUEST {scope?:str,operation:str,route:str,base_action:str}
RESPONSE baseline,result,retrieved,changed

```
› curl -s -X POST http://localhost:8000/api/subsequent -H "Content-Type: application/json" -d {"scope": "demo"}
```

GET /api/history/{scope} — backend/api/main.py → history()

REQUEST scope path
RESPONSE decisions,feedback

POST /api/gaps — backend/api/main.py → analyze()

REQUEST {text:str,scope?:str,values?:object}
RESPONSE id,capability,values,missing,blocking_frontier,questions,complete,edges

```
› curl -s -X POST http://localhost:8000/api/gaps -H "Content-Type: application/json" -d {"text": "Ship this as
  usual, but use the cheaper option."}
```

POST /api/gaps/{gid}/resolve — backend/api/main.py → resolve()

REQUEST {scope?:str,answers:object}
RESPONSE gap evaluation object

```
› curl -s -X POST http://localhost:8000/api/gaps/GAP_ID/resolve -H "Content-Type: application/json" -d {"scope":
  "demo"}
```

GET /api/graph/{scope} — backend/api/main.py → FieldRepository.graph()

REQUEST scope path
RESPONSE fields,relations,transitions

POST /api/reset — backend/api/main.py → FieldRepository.reset()

REQUEST {}
RESPONSE {reset:true}

```
› curl -s -X POST http://localhost:8000/api/reset -H "Content-Type: application/json" -d {"scope": "demo"}
```

04 DEMONSTRATION WORKFLOWS

§4 · THREE SOLUTIONS · RESET

◆ Step-by-step browser demonstrations of the three solutions and the global reset, with the exact inputs and the expected observations.

A Solution A

TAB · INTENT INSPECTOR

Open Intent Inspector → Keep default text → Click Inspect request → Observe result → Inspect field graph and transitions

Default text: Same shipment as last week, but two days earlier. Observe delivery_delta_days=-2, two historical candidates and unresolved historical_shipment_reference.

B Solution B

TAB · FEEDBACK ENGINE

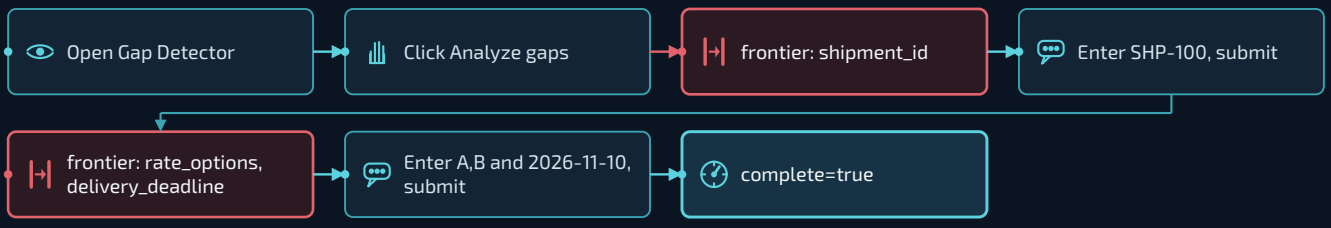
Open Feedback Engine → 1 Record decision → 2 Submit correction → 3 Confirm correction → 4 Subsequent decision → History

The confirmed quote/MUC-HAM correction changes the chosen action to Carrier B. The original decision remains in the decisions table.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

Solution C

TAB · GAP DETECTOR



Initial blocking frontier: shipment_id. Enter SHP-100 and submit. Next frontier: rate_options and delivery_deadline. Enter A,B and 2026-11-10; submit to reach complete=true.

RESET · GLOBAL DESTRUCTIVE

Reset all calls POST /api/reset and clears every row from six tables, across all scopes. It is a global destructive demo reset, not scoped. Existing SQLite file remains on disk.

05 EXACT TESTS, BUILD AND DEPLOYMENT

§5

SHELL

```
cd nemox-developer-tools
PYTHONPATH=backend python -m pytest -q backend/tests/test_system.py

cd frontend
npm install --no-audit --no-fund
npm run build

cd ..
docker compose up --build
```

RECORDED EVIDENCE FOR THIS HANDOVER

PATH	STATUS
Backend tests	5 passed
npm build	not run in this environment; node_modules absent
Docker	binary unavailable
Browser E2E	not run

VERIFICATION BOUNDARY

These are unverified runtime paths and are reported as such.

06 EXTENSION RECIPES

§6 · SIX RECIPES

- Concrete extension paths with the exact source locations to change and the tests to add.

Add field type

FieldRepository.create accepts arbitrary type strings. Add validation in backend/fieldcore/core.py before creating fields; update frontend FieldGraph labels and add test cases for schema and scope.

Add relation

Call FieldRepository.relation(source,target,kind,scope,event). Both fields must exist and share scope. Add a test verifying graph output and invalid scope behavior.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

Add API endpoint

Create a Pydantic model and route in backend/api/main.py, delegate to a domain service, add TestClient case in backend/tests/test_system.py, and add UI client call in frontend/src/services/api.ts.

Add capability

Edit CAPABILITIES in backend/gap_detector/service.py with required/questions/dependencies. Add a branch to capability(text), then tests for missing/frontier/complete and a cycle guard if introducing more complex dependencies.

Add feedback validity

Extend feedback correction schema with validity dates and operational identifiers. Modify retrieve() filtering and tests. Preserve original feedback rows and confirmed status.

Extend frontend

Extract sections of frontend/src/main.tsx into pages; preserve API JSON contracts and update the graph component for each solution. Add Vitest/Playwright before claiming browser acceptance.

07 SOURCE-LINKED ENGINEERING LIMITATIONS

§7

- No user/organization identity model
- Deterministic regex intent parser
- No shipment lookup API
- No full feedback relation coupling
- No database migrations
- No tenant authorization
- Synthetic history
- No general event stream
- Gap graph evaluates direct predecessor availability only
- Global reset

STATUS BOUNDARY

These are future extensions, not present behavior.

08 SOURCE AND DOCUMENTATION PARITY

§8 · MODULE · ACTUAL FUNCTION(S) · TEST

MODULE	ACTUAL FUNCTION(S)	TEST
README.md	configuration / UI	not directly covered
backend/Dockerfile	configuration / UI	backend/tests/test_system.py
backend/api/__init__.py	configuration / UI	backend/tests/test_system.py
backend/api/main.py	Request, Decision, Correction, Confirmation, Subsequent, Answers, invalid, health, intent, decision, feedback, confirmation, next_decision, decision_history, gaps, gap_resolution, graph, reset	backend/tests/test_system.py
backend/feedback_engine/__init__.py	configuration / UI	backend/tests/test_system.py
backend/feedback_engine/service.py	decide, correct, confirm, retrieve, subsequent, history	backend/tests/test_system.py
backend/fieldcore/__init__.py	configuration / UI	backend/tests/test_system.py
backend/fieldcore/core.py	now, uid, connection, init, FieldRepository, FieldContextResolver, create, get, transition, relation, graph, reset, resolve	backend/tests/test_system.py
backend/gap_detector/__init__.py	configuration / UI	backend/tests/test_system.py
backend/gap_detector/service.py	capability, analyze, evaluate, resolve	backend/tests/test_system.py
backend/intent_inspector/__init__.py	configuration / UI	backend/tests/test_system.py
backend/intent_inspector/service.py	inspect	backend/tests/test_system.py
backend/requirements.txt	configuration / UI	backend/tests/test_system.py
backend/tests/test_system.py	clean, test_intent_trace, test_feedback_scope_and_confirm, test_gap_resolution, test_state_validation, test_relation_scope	backend/tests/test_system.py
docker-compose.yml	configuration / UI	not directly covered

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

MODULE	ACTUAL FUNCTION(S)	TEST
frontend/Dockerfile	configuration / UI	not directly covered
frontend/index.html	configuration / UI	not directly covered
frontend/package.json	configuration / UI	not directly covered
frontend/src/components/FieldGraph.tsx	configuration / UI	not directly covered
frontend/src/main.tsx	configuration / UI	not directly covered
frontend/src/services/api.ts	configuration / UI	not directly covered
frontend/src/style.css	configuration / UI	not directly covered
frontend/tsconfig.json	configuration / UI	not directly covered
frontend/vite.config.ts	configuration / UI	not directly covered
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/Dockerfile	configuration / UI	not directly covered
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/Dockerfile	configuration / UI	not directly covered
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/Dockerfile	configuration / UI	not directly covered
legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/docker-compose.yml	configuration / UI	not directly covered
start.sh	configuration / UI	not directly covered

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

LEGAL NOTICE | CONFIDENTIALITY | RESERVATION OF RIGHTS

BINDING FOR EVERY RECIPIENT, READER OR HOLDER OF THIS DOCUMENT

1 Ownership

This document — including its structure, SYNTX representation, architecture reconstruction, recursive and inverse sweep results, diagrams, status classification and all derived descriptions — is the confidential intellectual work of Ottavio Braun, SYNTX SYSTEM. All rights are reserved.

2 No licence

No right of use of any kind is granted. Receipt, possession, access to or knowledge of this document does not constitute a licence, transfer of rights, offer, order, contract, partnership or commitment of any kind.

3 Written approval required

Any use, reproduction, storage beyond inspection, distribution, disclosure to third parties, adaptation, translation, implementation, re-engineering, integration into products or services, and any use for training, evaluating or prompting AI systems — in whole or in part, directly or indirectly — requires the prior, explicit, written approval of Ottavio Braun. Approval is valid only if confirmed by Ottavio Braun personally, in text form, for a named purpose and recipient. Silence, conduct, payment or prior collaboration do not constitute approval.

4 Purpose limitation

Until such approval is given, this document may be read solely to evaluate whether a written agreement with Ottavio Braun is to be concluded. Any other purpose is excluded.

5 Return and deletion

On request, and in any case if no written approval is granted, all copies — physical and digital, including excerpts, notes and derived material — must be deleted or returned without delay and their deletion confirmed in writing.

6 No warranty

The document describes an inspected source-code state as of 1 November 2026. It makes no representation of completeness, correctness, fitness for a particular purpose, security or production readiness. Implemented, derived, conceptual and proposed elements are distinguished; none of them is warranted.

7 No liability

No liability is accepted for decisions, implementations or damages arising from use of this document without the written approval described in section 3.

8 Third-party names

nemox, FastAPI, React, Vite, SQLite, Docker, React Flow / @xyflow and other product names remain the property of their respective owners and are used solely to describe the inspected implementation.

9 Proprietary method

The SYNTX Recursive Deep Sweep and Inverse Sweep are proprietary procedures of Ottavio Braun. Their internal material is not reproduced and no right in them is conveyed by this document.

Deutsch · verbindlicher Hinweis

Vertraulich. Jede Nutzung, Vervielfältigung, Weitergabe, Bearbeitung oder Umsetzung dieses Dokuments — ganz oder teilweise — bedarf der vorherigen, ausdrücklichen, schriftlichen Freigabe durch Ottavio Braun. Ohne diese Freigabe wird kein Nutzungsrecht eingeräumt.

APPROVAL CONTACT
Ottavio Braun | SYNTX SYSTEM
mirror@syntx-system.com

MUNICH, NOVEMBER 2026

© 2026 OTTAVIO BRAUN · ALL RIGHTS RESERVED